

Algoritmo de Dijkstra

O **Algoritmo de Dijkstra** é um algoritmo utilizado para encontrar o **menor caminho** entre um vértice de origem e todos os outros vértices de um grafo que possua **pesos positivos**.

Algoritmo de Dijkstra

Entendendo de forma simples

Imagine que você está em uma cidade e quer descobrir qual é o caminho mais curto para chegar a vários bairros.

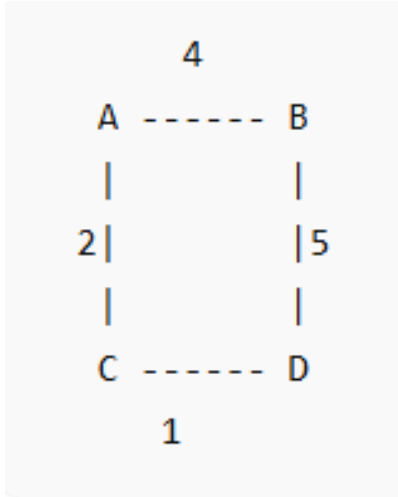
O Dijkstra funciona da seguinte forma:

1. Começa no ponto de origem.
2. Marca a distância da origem para ela mesma como **0**.
3. Marca todas as outras distâncias como **infinito**.
4. Escolhe o vértice não visitado com a menor distância conhecida.
5. Atualiza as distâncias dos seus vizinhos.
6. Repete até visitar todos os vértices.

Algoritmo de Dijkstra

Exemplo

Considere o seguinte grafo:



E uma ligação adicional:

A ----- 7 ----- D

Algoritmo de Dijkstra

Representação completa:

A -> B = 4

A -> C = 2

A -> D = 7

B -> D = 5

C -> D = 1

Queremos descobrir o menor caminho saindo de **A**.

Algoritmo de Dijkstra

Passo 1

Distâncias iniciais:

Vértice	Distância
A	0
B	∞
C	∞
D	∞

Visitados: nenhum.

Algoritmo de Dijkstra

Passo 2

Escolhemos A (distância 0).

Analisamos seus vizinhos:

A $\rightarrow$ B = 4

A $\rightarrow$ C = 2

A $\rightarrow$ D = 7

Atualizamos:

Vértice	Distância
A	0
B	4
C	2
D	7

A torna-se visitado.

Algoritmo de Dijkstra

Passo 3

O menor não visitado é C (2).

Analizamos:

$$C \rightarrow D = 1$$

Novo custo até D:

$$A \rightarrow C \rightarrow D$$

$$2 + 1 = 3$$

Algoritmo de Dijkstra

Como $3 < 7$:
Atualizamos D.

Vértice	Distância
A	0
B	4
C	2
D	3

C visitado.

Algoritmo de Dijkstra

Passo 4

O menor não visitado é D (3).

D não melhora nenhum caminho.

D visitado.

Algoritmo de Dijkstra

Passo 5

Resta B (4).

$B \rightarrow D = 5$

$4 + 5 = 9$

Como $9 > 3$, não atualizamos.

B visitado.

Algoritmo de Dijkstra

Resultado Final

Menores distâncias a partir de A:

Destino	Distância
A	0
B	4
C	2
D	3

Caminhos:

$A \rightarrow B = 4$

$A \rightarrow C = 2$

$A \rightarrow C \rightarrow D = 3$

Algoritmo de Dijkstra

```
#include <stdio.h>
#include <limits.h>
```

```
#define V 4
```

```
//== Para simplificar, utilizaremos uma matriz de adjacência.
```

```
int minDistancia(int dist[], int visitado[]) {
    int min = INT_MAX;
    int minIndice;

    for (int v = 0; v < V; v++) {
        if (!visitado[v] && dist[v] <= min) {
            min = dist[v];
            minIndice = v;
        }
    }

    return minIndice;
}
```

```

void dijkstra(int grafo[V][V], int origem) {
    int dist[V];
    int visitado[V];
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visitado[i] = 0;
    }
    dist[origem] = 0;
    for (int cont = 0; cont < V - 1; cont++) {
        int u = minDistancia(dist, visitado);
        visitado[u] = 1;
        for (int v = 0; v < V; v++) {
            if (!visitado[v] &&
                grafo[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + grafo[u][v] < dist[v]) {

                dist[v] = dist[u] + grafo[u][v];
            }
        }
    }
    printf("Menores distancias a partir da origem:\n");
    for (int i = 0; i < V; i++) {
        printf("Vertice %d -> %d\n", i, dist[i]);
    }
}

```

Algoritmo de Dijkstra

```
int main() {  
  
    int grafo[V][V] = {  
        {0, 4, 2, 7},  
        {4, 0, 0, 5},  
        {2, 0, 0, 1},  
        {7, 5, 1, 0}  
    };  
  
    dijkstra(grafo, 0);  
  
    return 0;  
}
```

Algoritmo de Dijkstra